

人工智能数学原理与算法 B

第三次实验

逻辑回归与大学排名分类

分阶段实验手册

主讲 钟志诚

整理 曾有成

单位 中国科学技术大学

人工智能与数据科学学院

从数据到可解释的分类流程

数据清洗与划分 → 数值表示 → 线性得分与概率 → 交叉熵与正则化 → 梯度更新 → 独立评估。

2026 年秋季 建议 3 学时 (135 分钟)

C0-C7 分阶段完成；日期与提交平台以课程通知为准。

随包数据为 QS (Quacquarelli Symonds) 2026 排名的教学整理版。

目录

1 实验入口与完成目标	1
2 课堂路线与验收方式	2
3 C0–C1：认识表格并清洗	3
4 C2：数值与类别分别处理	4
5 C3–C4：从样本到模型	5
6 C5–C6：目标与一次更新	6
7 C7：选择、解释与最终测试	7
8 选做：特征选择对照	8
9 实验报告与提交	9
10 资料与改编说明	11

1 实验入口与完成目标

1.1 先读什么，再运行什么

课前阅读《前置讲义》第 1-5 节，复习表格、填补、独热编码和批次，约 30 分钟。课堂以数学推导、手算和检查点完成训练与评价。主讲钟志诚，整理曾有成；建议课堂 3 学时，共 135 分钟，安装时间不计入。

实验用 QS (Quacquarelli Symonds) 2026 排名整理表，识别同年榜单前 500 名；不预测未来排名。沿用实验 01 环境，在 `code/` 目录执行：

```
conda activate ustc-ai-experiments-26fall-B
python checkpoints.py C0
```

缺少依赖时运行 `python -m pip install -r requirements.txt`。使用 CPU (Central Processing Unit，中央处理器) 即可；不要求加速硬件。本包不依赖 `scikit-learn`。

入口	作用
<code>student.py</code>	主要补全文件，17 处 TODO 均有 Hint
<code>checkpoints.py</code>	C0-C7 小例子自查，可单独执行
<code>support.py</code> , <code>workflow.py</code>	已提供的预处理编排、选择与文件输出
<code>run_lab.py</code> , <code>logic.ipynb</code>	两种入口共用同一份学生实现
<code>data/classroom/</code>	固定课堂数据、来源与划分记录
<code>report-template.md</code>	逐检查点记录及综合结果模板

1.2 统一练习节奏

先运行已完成小例子，观察并修改一个值；再补一个函数、运行自检、解释结果。Notebook 通过普通 `import student` 复用代码；修改文件后重启内核，不在两处维护答案。`return None` 是待完成占位，不是正确算法输出。

先明确输入边界

模型只用三个数值分数与五个类别字段。名称、行号、排名、总分、历史排名不输入。类别缺失保留为 `Unknown`，数值缺失由训练均值处理；不能用测试集决定填补规则或删除哪些字段。

2 课堂路线与验收方式

2.1 135 分钟安排

阶段/分	教师活动	学生活动与产出
C0 / 10	从决策树连接概率输出，演示形状	运行矩阵乘法，记录改值后的预测
C1 / 15	区分标签、标识与数值缺失	补两个清洗函数，报告字段与缺失
C2 / 25	手算标准化与独热编码，说明未知类别	补五个函数，核对列顺序和张量
C3 / 10	解释样本成对与末批	补数据集和 loader，用五条数据自检
C4 / 15	连接得分、概率与预测类别	补参数和前向，核对得分 5
C5 / 20	推导交叉熵与权重惩罚	补损失、正则，验证截距不受罚
C6 / 20	手算一次更新，解释五步循环	补训练，核对连续两次更新
C7 / 20	区分选择和测试，解释错误代价	补指标、比较方案，冻结后测试

合计 $10 + 15 + 25 + 10 + 15 + 20 + 20 + 20 = 135$ 分钟。运行完整训练时同步分析图表；特征选择对照为课后选做，不挤占必做课堂时间。

2.2 依赖与证据

C0 独立可运行；C1 清洗数值和标签；C2 的数值与类别小测试独立于真实数据；C3 使用提供的张量；C4 建模；C5 需要 C4；C6 依赖 C3–C5；C7 指标小例子独立，完整实验需要全部阶段。

检查点验收

每个阶段记录对应函数、实现思路、一条小例子的输入与预期/实际输出及解释。**PASS** 只能证明所列样例通过，不能代替报告中的分析；未完成项如实说明。

2.3 必做与选做

必做：C0–C7、一次梯度手算、三种正则方案比较、冻结后测试及四部分报告。选做：只在训练与验证上比较完整/精简特征。两者不以高准确率作为评分门槛。

板书安排

左：数据角色与编码表；中： $\mathbf{X}\theta \rightarrow \mathbf{p} \rightarrow R \rightarrow \nabla R$ （设计矩阵、参数、概率、平均损失、梯度）；右：训练/验证/测试职责和四种分类计数。详细维度与记号见前置讲义。

3 C0–C1: 认识表格并清洗

3.1 C0: 运行形状小例子

运行 `python checkpoints.py C0`。训练原表应为 (902,11)，缺失标签 0，矩阵乘法为 $\begin{bmatrix} 2.0 \\ -1.0 \end{bmatrix}$ 。11 列含名称、行号、标签，不能理解成 11 个模型输入。改变权重后先手算，再运行核对。

3.2 C1: 补两个简洁函数

在 `student.py` 补 `filter_labels` 和 `clean_numeric`。前者只删除标签缺失行，不填补特征；后者把数值字符串转换成数值，空白、横线和无法解析文本转成缺失，不填 0。

```
import support
train = support.load_split(support.DATA_DIR, "train")
print(train.shape)
print(train.isna().sum())
print(train["top500"].value_counts())
print(train["status"].value_counts(dropna=False))
```

运行 `python checkpoints.py C1`。小例子三条记录的标签为 1、缺失、0，结果保留两条；第一条的特征仍然缺失。数值例子 `["12.5", "-", "", None]` 转换后只有第一项是 12.5。

3.3 明确哪些信息不能输入

课堂字段 `region`、`status`、`size`、`focus`、`research` 是类别；三个以 `score` 结尾的字段为分数。`name` 与 `row_id` 只追溯来源；`top500` 是标签。源排名按实际名次和并列关系生成标签，有 502 条正类，不按前 500 行生成。

三份规模及正类数分别为 902/301、300/100、302/101。真实缺失已保留，不需要人工加缺失。数据构造脚本由教师提供，不是学生另写一遍的任务；原镜像与规则见数据说明。

报告中记录

列出八个输入字段及其类型，说明为何原行号也要排除；记录训练缺失统计。标签缺失处理用小例子证明，不能伪造真实数据中不存在的标签缺失。

4 C2: 数值与类别分别处理

4.1 数值分支: 三个函数

补 `fit_statistics`、`apply_statistics`、`to_tensors`。先算训练非缺失均值并填充, 再算总体标准差; 转换验证/测试时不重算。最后转换为浮点张量, 首列加常数 1, 标签成为列向量。

数值核对

训练 [1,3,缺失]: 均值 2, 标准差约 0.816497; 验证 100 的标准化值约 120.025。均值不会因为验证值很大而改变。常数列尺度改为 1, 全空训练列均值约定为 0。

4.2 类别分支: 两个函数

补 `fit_categories` 和 `encode_categories`。前者逐列取训练中出现的非缺失值、排序, 最后添加 Unknown; 后者按固定类别表逐列生成 0/1 指示。缺失与未见类别统一归入 Unknown。

小例子训练取值为 Asia、Europe、缺失, 列序为 Asia、Europe、Unknown; 验证取值 Europe、Atlantis、缺失, 预期三行分别为 (0,1,0)、(0,0,1)、(0,0,1)。这里 Atlantis 是用于检查未知值处理的虚构类别, 不是源数据地区。

```
import student
train, val, state = support.prepare_train_val(
    student, support.DATA_DIR)
print(train[0].shape, train[1].shape)
print(val[0].shape, val[1].shape)
print(state["categories"])
```

运行 `python checkpoints.py C2`。完整特征为三个标准化数值与 25 个独热列, 训练输出 (902,29)、(902,1), 验证输出 (300,29)、(300,1)。新增首列常数包含在 29 中, 测试文件尚未读取。

报告中记录

各给一条数值填补和独热编码证据; 解释未知类别、列顺序与首列常数。不要在验证/测试上单独调用类别拟合, 也不要标准化标签或独热列。

5 C3–C4: 从样本到模型

5.1 C3: 保持特征与标签成对

补 `UniversityDataset.__len__`、`__getitem__` 与 `make_loader`。构造函数已保存 `X` 和 `y`；每个索引返回对应的一对张量。只在训练打乱，保留不足批量的最后一批。

```
dataset = student.UniversityDataset(*train)
loader = student.make_loader(dataset, 64, True)
for X_batch, y_batch in loader:
    print(X_batch.shape, y_batch.shape)
```

运行 `python checkpoints.py C3`。五条样本、每批两条得到 2,2,1；真实训练 902 条、每批 64 条得到 15 批，末批 6 条。标签不单独洗牌。

5.2 C4: 线性得分和概率

补 `sigmoid`，以及模型类的 `__init__`、`forward`。参数是一个列向量，行数等于增广维数，用 `nn.Parameter` 注册并零初始化。前向返回矩阵乘积；`sigmoid` 在概率计算时单独调用。

```
model = student.LogisticRegression(train[0].shape[1])
logits = model(train[0][:3])
probabilities = student.sigmoid(logits)
print(logits.shape, probabilities)
```

运行 `python checkpoints.py C4`。参数 $(1, 2, -1)^\top$ 、增广输入 $(1, 3, 2)^\top$ 的得分是 5，概率约 0.993307；不能把概率作为前向得分返回。

报告中记录

C3 记录成对取样与末批；C4 记录输入、参数、得分、概率和形状。说明 `@` 与逐元素乘法 `*` 的区别。

6 C5–C6: 目标与一次更新

6.1 C5: 二元交叉熵和权重惩罚

补 `data_loss` 和 `penalty`。采用二元交叉熵 BCE (Binary Cross-Entropy)，由 `BCEWithLogitsLoss` 接收原始得分并稳定计算损失。

L1 (L1 Norm, 一范数) 惩罚为特征权重绝对值和；L2 (L2 Norm, 二范数) 惩罚为二范数平方的一半。两者只用 `theta[1:]`，不惩罚截距。

运行 `python checkpoints.py C5`。零得分时平均交叉熵约 0.693147；参数 (10, 3, -4) 的两种惩罚分别为 7、12.5；L2 惩罚对截距的梯度是零。

6.2 C6: 补五步训练循环

补 `train_epoch`：清梯度、前向、构造数据损失加惩罚的总目标、反向传播、更新参数。SGD (Stochastic Gradient Descent, 随机梯度下降) 的自动微分与参数更新器已提供，学生补全连接步骤。

```
optimizer.zero_grad()
logits = model(X)
loss = data_loss(logits, y)
objective = loss + strength * model.penalty(kind)
objective.backward()
optimizer.step()
```

训练时保留计算图，不能在反向前调用 `item()` 或 `detach()`。每个批次都清梯度；本函数返回的是各批更新前数据损失的加权平均，绘图另用每轮结束的完整数据损失。

手算再运行

两所假设学校的增广行是 (1, 1, 0)、(1, 0, 1)，标签 1、0。零参数时概率均为 0.5，平均梯度为 $(0, -0.25, 0.25)^\top$ 。学习率 0.4、无正则，更新为 $(0, 0.1, -0.1)^\top$ 。详见前置讲义第 8 节。

运行 `python checkpoints.py C6`。自检还核对第二次更新，用来发现忘记清梯度的问题。

报告中记录

C5 记录损失与两种惩罚的数值；C6 记录一次手算的输入、概率、梯度、更新及代码对照。只贴“损失下降”不足以证明更新正确。

7 C7: 选择、解释与最终测试

7.1 先在验证集选模型

补 `binary_metrics`, 运行小样例和全部自检, 然后训练:

```
python checkpoints.py all
python run_lab.py
```

默认输出 `code/outputs-qs/`。零初始化、训练种子 42、批量 64、学习率 0.05、最多 60 轮; 比较无正则、L2 系数 0.01、L1 系数 0.01。每组取验证数据损失最低轮, 再从三组中选择最低者, 阈值预先固定为 0.5。

- `candidates.csv`: 三方案的设置与验证结果。
- `none_loss.png`、`l2_loss.png`、`l1_loss.png`: 同一参数下的训练/验证数据损失, 不包含惩罚项。
- `selection.json`、`preprocessing.json`、`selected.pt`: 冻结方案、训练预处理和模型权重。
- `validation_thresholds.csv`: 阈值 0.3/0.5/0.7 的验证计数与指标, 只用于权衡讨论。
- `weights.csv`: 对应特征名的参数。L1 不保证精确零权重。

7.2 记录选择后再测试

```
python run_lab.py --final-test
```

恢复冻结方案, 只使用保存的均值、尺度和类别表, 输出 `test_metrics.json` 与 `test_predictions.csv`。训练多数类基线总预测负类。

TP (True Positive, 真正例)、FP (False Positive, 假正例)、FN (False Negative, 假负例)、TN (True Negative, 真负例) 以前 500 名为正类。报告准确率、精确率、召回率、F1 分数 (F1 Score) 和混淆矩阵; 分母为零按 0 处理。矩阵行是真实类别, 列是预测类别。

报告中记录

四条小例子标签 `[1, 0, 1, 0]`、概率 `[0.9, 0.8, 0.4, 0.1]` 在阈值 0.5 时四种计数各 1; 解释降低阈值后的变化。综合实验记录选定方案、一张曲线、一张混淆矩阵, 不用单一准确率下结论。

8 选做：特征选择对照

8.1 改变一个因素

在查看测试结果之前，可执行：

```
python run_lab.py --feature-study
```

完整组使用三个数值与五个类别字段；精简组只删除 focus、research。保留同一数据划分，固定 L2 系数 0.01、学习率 0.05、批量 64、训练种子 42、最多 60 轮，每组按验证损失取最佳轮。维数由 28 降为 18，增广后分别 29、19。

输出在 `outputs-qs/feature_study/comparison.csv`。选做不读取测试集、不改变主实验冻结模型，不把测试结果反复用于字段选择。它检验的是整组字段的作用，不是逐一证明每个字段都重要。

8.2 报告怎样解释

列出保留/删除字段、维数、验证损失、准确率、精确率、召回率与 F1 分数 (F1 Score)。如果精简后变差，说明当前规则删去了有用信息；如果相近，可讨论复杂度与可解释性的取舍。不要声称“特征越少越好”或“正则化一定提高准确率”。

两个常见混淆

删除泄漏字段是建模前必须做的筛选，不能当成可自由调节的性能技巧。L1 惩罚鼓励稀疏，但当前普通梯度更新不保证精确零权重；不能直接把小权重视为完整字段已经被删除。

8.3 课堂追问与答案要点

1. 为什么学校规模不直接设为 1、2、3、4？这样会假定相邻档位对得分的作用等距；独热编码放松该假设。
2. 为什么不按全部数据收集类别？模型表示也属于训练过程，应预先处理验证/测试中的未知类别。
3. 两个方案准确率相同，能否认为完全等价？不能，概率损失和四种计数可能不同。
4. 为什么不直接用总分？当前目标由排名构造，总分近乎直接提供目标信息。

9 实验报告与提交

9.1 四部分正文

提交 `report.pdf`, PDF (Portable Document Format, 可移植文档格式) 首页列实验名称、学号、姓名。参照模板写四部分：

- 1. **实验概况**。任务与适用范围, C0–C7 完成状态, 未解决问题。
- 2. **逐检查点实现与验证**。对应函数、思路和关键代码、小例子输入与预期/实际输出、结果解释。每项通常摘录 3–10 行关键代码, 不贴整份文件; C2 同时覆盖数值与类别, C6 包含手算。
- 3. **综合实验结果**。来源、划分、类别比例、缺失与编码维数、训练设置、三方案验证表、测试前选择理由、一张损失曲线、最终指标和混淆矩阵。选做对照单独列验证结果。
- 4. **结果分析与总结**。解释误报/漏判、基线、至少两个验证阈值、正则化是否改善; 说明同年分类、镜像来源、非因果与跨年泛化局限, 写收获或建议。

9.2 每一检查点至少记录什么

阶段	一项可复核证据
C0	原例与修改权重后的矩阵乘法; 说明形状
C1	含缺失标签的三行例子及清洗数值字符串结果
C2	均值/尺度/验证值; 已知、缺失、未知类别的独热行
C3	第 3 条样本与标签配对, 五条数据得到 2, 2, 1
C4	给定参数与输入的得分、概率、维度
C5	零得分损失, 两种惩罚与截距处理
C6	概率、梯度、一次更新及手算/程序对照
C7	四种计数的小例子; 综合部分引用实际完整运行

同一综合图表只放一次, 逐检查点处可以引用。所有数值来自自己的提交代码; 未完成项如实标注, 不能用教师参考数值冒充个人运行。

9.3 提交结构与评分建议

提交 学号 _ 姓名 _lab03.zip, 含 report.pdf 与 code/: 学生实现、运行入口、support、workflow、checkpoints、依赖文件, 以及 data/classroom/ 三份数据和划分说明。使用 Notebook 时同时保留 logic.ipynb 和 student.py。

无需打包环境、缓存、原始大表、教师答案或课程讲义; 输出图表放入报告, 模型权重与预测明细可自行保留。截止时间和提交平台以课程通知为准。

建议比例	核对内容
数据准备 30%	数值清洗、训练统计量、独热编码、未知值与列顺序
模型实现 40%	参数与前向、概率接口、数据损失和两种惩罚
训练评价 30%	梯度更新、验证选择、最终测试及解释与反馈

这是对原指导书评分结构的教学适配, 课程另行通知优先。选做对照用于拓展展示, 不设必达准确率, 不按截图数量或篇幅评分。

9.4 提交前自查与排错

- 全部 checkpoint 通过, 报告中的小例子能复算。解压到其他位置后 C0 可直接运行。
- 两个运行命令使用同一数据目录和输出目录; 更换数据时使用新输出目录。
- 维数不一致: 检查未知类别是否新增列、拼接顺序是否变化、首列常数是否遗漏。
- 损失异常: 检查数值缺失、尺度、重复 sigmoid、学习率以及反向前切断计算图。
- 第二次手算不一致: 检查每批是否清梯度。
- 训练总目标与曲线不同: 曲线是无正则数据损失, 不是带惩罚的目标。
- 测试很高不代表任务正确: 先核对是否混入排名、总分或排序行号。

9.5 参考运行与解释

本环境固定划分与设置下, 三组均在第 60 轮取得各自最低验证损失, 无正则约 0.371792, L2 约 0.379552, L1 约 0.392744, 三组验证准确率均 0.83。故主实验选择无正则, 而非预设正则化一定更好。

冻结后一次测试: 准确率 84.11%、精确率 81.18%、召回率 68.32%、F1 约 0.7419; 混淆矩阵为真负例 185、假正例 16、假负例 32、真正例 69。多数类基线为 66.56%。这是本次数据的实际运行, 不是教材结果或未来年份效果。

10 资料与改编说明

10.1 相较原指导书保留和改变的部分

保留第一章逻辑回归的概率、损失、正则化、训练与评价主线；贷款数据改为 QS 大学数据，去掉日期和就业年限编码，加入数值缺失与独热编码、Unknown 和列对齐。增广首列表示截距，前向返回得分，`BCEWithLogitsLoss` 接收得分。

本次有 1504 所学校、8 个原始输入字段，分层划分为 902/300/302。不得继续使用旧版 8000/1000/1000、贷款类别比例或召回率。此前贷款方案保留用于追溯，不在当前学生包内。

10.2 实际引用资料

- 项目上级目录《人工智能数学原理与算法：实验指导书（最终）.pdf》第 1 章，第 1.4 节对应清洗、第 1.5 节对应模型与训练、第 1.6 节对应评分结构。
- 同级实验 01 的实验手册和报告模板：提交组织；实验 02 的代码：检查点风格。
- 前置讲义列出的课程总表、逻辑回归课件及数学记号说明。
- `code/data/README.md`、原表和划分摘要：下载来源、字段映射、标签与真实缺失；`teacher/prepare_classroom_data.py`：可复现的课堂划分。
- [QS 官方学校分类说明](#)；[QS 排名方法说明](#)，核对日期 2026-10-08。

报告的逐任务组织沿用此前用户提供的《数据科学基础》HW1 个人成稿中“任务、代码、输出、结论”的思路；不沿用其题目、结果、截止日期或提交渠道。教师参考和验收记录见 `teacher/README.md`，学生包不含答案。

10.3 结束时应能独立回答

为什么编码表也是需要拟合的状态？为什么排名行号不能输入模型？为什么正则化比较要用同一种验证损失？为什么一次划分表现良好不意味着能够预测下一年的大学排名？把这四个问题的答案写成各一句话，作为实验退出卡。